

Week 6: Product Review and Rating Management System

Scenario:

The Product Review and Rating Management System is used to collect customer feedback about purchased products. Customers can provide ratings and reviews after completing their orders. The system stores customer opinions, analyzes product performance, and identifies highly rated products.

1. Create Review and Rating Tables

```
SQL> CREATE TABLE REVIEW(  
2     ReviewID VARCHAR2(7),  
3     CustomerID NUMBER,  
4     ProductID NUMBER,  
5     Rating NUMBER(1),  
6     ReviewText VARCHAR2(50),  
7     ReviewDate DATE,  
8     PRIMARY KEY(ReviewID),  
9     FOREIGN KEY(CustomerID) REFERENCES CUSTOMER(CUSTOMER_ID),  
10    FOREIGN KEY(ProductID) REFERENCES PRODUCT(PRODUCT_ID)  
11 );
```

Table created.

```
SQL> DESC REVIEW;
```

Name	Null?	Type
REVIEWID	NOT NULL	VARCHAR2(7)
CUSTOMERID		NUMBER
PRODUCTID		NUMBER
RATING		NUMBER(1)
REVIEWTEXT		VARCHAR2(50)
REVIEWDATE		DATE

Inserting a values and Store Customer Feedback and Ratings

```
SQL> INSERT INTO REVIEW  
2 VALUES ('REV0001', 201, 101, 5, 'Excellent product', TO_DATE('20-09-2026','DD-MM-YYYY'));  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW  
2 VALUES ('REV0002', 202, 102, 4, 'Good quality', TO_DATE('21-09-2026','DD-MM-YYYY'));  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW  
2 VALUES ('REV0003', 203, 103, 5, 'Very nice frame', TO_DATE('22-09-2026','DD-MM-YYYY'));  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW  
2 VALUES ('REV0004', 204, 104, 3, 'Average quality', TO_DATE('23-09-2026','DD-MM-YYYY'));  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW  
2 VALUES ('REV0005', 205, 105, 4, 'Good product', TO_DATE('24-09-2026','DD-MM-YYYY'));  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW  
2 VALUES ('REV0006', 206, 106, 5, 'Excellent quality', TO_DATE('25-09-2026','DD-MM-YYYY'));  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW  
2 VALUES ('REV0007', 207, 107, 4, 'Worth the price', TO_DATE('26-09-2026','DD-MM-YYYY'));  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW  
2 VALUES ('REV0008', 208, 108, 2, 'Needs improvement', TO_DATE('27-09-2026','DD-MM-YYYY'));  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW  
2 VALUES ('REV0009', 209, 109, 5, 'Beautiful frame', TO_DATE('28-09-2026','DD-MM-YYYY'));  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO REVIEW  
2 VALUES ('REV0010', 210, 110, 4, 'Satisfied with product', TO_DATE('29-09-2026','DD-MM-YYYY'));  
  
1 row created.
```

```
SQL> SELECT * FROM REVIEW;
```

REVIEWID	CUSTOMERID	PRODUCTID	RATING	REVIEWTEST	REVIEWDAT
REV0001	201	101	5	Excellent product	20-SEP-26
REV0002	202	102	4	Good quality	21-SEP-26
REV0003	203	103	5	Very nice frame	22-SEP-26
REV0004	204	104	3	Average quality	23-SEP-26
REV0005	205	105	4	Good product	24-SEP-26
REV0006	206	106	5	Excellent quality	25-SEP-26
REV0007	207	107	4	Worth the price	26-SEP-26
REV0008	208	108	2	Needs improvement	27-SEP-26
REV0009	209	109	5	Beautiful frame	28-SEP-26
REV0010	210	110	4	Satisfied with product	29-SEP-26

10 rows selected.

Store customer ratings from 1 to 5 stars.

Ensure this by adding Constraint

```
SQL> ALTER TABLE REVIEW
  2 ADD CONSTRAINT CHK_REVIEW_RATING
  3 CHECK (Rating >=1 AND Rating <=5);

Table altered.
```

```
SQL> UPDATE REVIEW
  2 SET ReviewTest = 'Good quality'
  3 WHERE ReviewID = 'REV0004';

1 row updated.

SQL> SELECT * FROM REVIEW
  2 WHERE ReviewID = 'REV0004';
```

REVIEWID	CUSTOMERID	PRODUCTID	RATING	REVIEWTEST	REVIEWDAT
REV0004	204	104	3	Good quality	23-SEP-26

Update review comments if required.

```
SQL> DELETE FROM REVIEW
  2 WHERE ReviewID = 'REV0008';

1 row deleted.
```

Remove inappropriate or invalid reviews.

Retrieve Product Review Details

1. Display all reviews for a product.

```
SQL>
SQL> SELECT *
  2  FROM REVIEW
  3  WHERE ProductID = 101;
```

REVIEWID	CUSTOMERID	PRODUCTID	RATING	REVIEWTEST	REVIEWDAT
REV0001	201	101	5	Excellent product	20-SEP-26

2. Display customer name with their reviews.

```
SQL> SELECT C.CUSTOMER_NAME,
  2         R.ProductID,
  3         R.Rating,
  4         R.ReviewTest,
  5         R.ReviewDate
  6 FROM CUSTOMER C
  7 JOIN REVIEW R
  8 ON C.CUSTOMER_ID = R.CustomerID;
```

CUSTOMER_N	PRODUCTID	RATING	REVIEWTEST	REVIEWDAT
ANU	108	2	Needs improvement	27-SEP-26
ARUN	101	5	Excellent product	20-SEP-26
PRIYA	102	4	Good quality	21-SEP-26

CUSTOMER_N	PRODUCTID	RATING	REVIEWTEST	REVIEWDAT
KARTHIK	103	5	Very nice frame	22-SEP-26
DIVYA	104	3	Good quality	23-SEP-26
SURESH	105	4	Good product	24-SEP-26

CUSTOMER_N	PRODUCTID	RATING	
KARTHIK	103	5	
Very nice frame			22-SEP-26
DIVYA	104	3	
Good quality			23-SEP-26
SURESH	105	4	
Good product			24-SEP-26

CUSTOMER_N	PRODUCTID	RATING	
MEENA	106	5	
Excellent quality			25-SEP-26
VIGNESH	107	4	
Worth the price			26-SEP-26
RAHUL	109	5	
Beautiful frame			28-SEP-26

CUSTOMER_N	PRODUCTID	RATING	
NISHA	110	4	
Satisfied with product			29-SEP-26

10 rows selected.

3. Find products having maximum reviews.

```
SQL> SELECT P.PRODUCT_ID,
2      P.PRODUCT_NAME,
3      COUNT(R.REVIEWID) AS TOTAL_REVIEWS
4  FROM PRODUCT P
5  JOIN REVIEW R
6  ON P.PRODUCT_ID = R.PRODUCTID
7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
8  ORDER BY TOTAL_REVIEWS DESC;
```

PRODUCT_ID	PRODUCT_NAME	TOTAL_REVIEWS
104	Decorative Wall Frame	1
109	Designer Wall Frame	1
110	Classic Table Frame	1
108	Large Collage Frame	1
106	Modern Photo Frame	1
107	Premium Acrylic Frame	1
102	Elegant Acrylic Frame	1
103	Family Collage Frame	1
105	Table Photo Frame	1
101	Classic Wooden Frame	1

10 rows selected.

4. Display recent customer feedback.

```
SQL> SELECT C.CUSTOMER_NAME,
2         P.PRODUCT_NAME,
3         R.RATING,
4         R.REVIEWTEST AS CUSTOMER_FEEDBACK,
5         R.REVIEWDATE
6 FROM CUSTOMER C
7 JOIN REVIEW R
8 ON C.CUSTOMER_ID = R.CUSTOMERID
9 JOIN PRODUCT P
10 ON R.PRODUCTID = P.PRODUCT_ID
11 ORDER BY R.REVIEWDATE DESC;
```

CUSTOMER_N	PRODUCT_NAME	RATIN

CUSTOMER_FEEDBACK		REVIEWDAT

NISHA	Classic Table Frame	
Satisfied with product		29-SEP-26
RAHUL	Designer Wall Frame	
Beautiful frame		28-SEP-26
ANU	Large Collage Frame	
Needs improvement		27-SEP-26

CUSTOMER_N PRODUCT_NAME		RATIN

CUSTOMER_FEEDBACK		REVIEWDAT

VIGNESH	Premium Acrylic Frame	
Worth the price		26-SEP-26
MEENA	Modern Photo Frame	
Excellent quality		25-SEP-26
SURESH	Table Photo Frame	
Good product		24-SEP-26

CUSTOMER_N PRODUCT_NAME		RATIN

CUSTOMER_FEEDBACK		REVIEWDAT

DIVYA	Decorative Wall Frame	
Good quality		23-SEP-26
KARTHIK	Family Collage Frame	
Very nice frame		22-SEP-26
PRIYA	Elegant Acrylic Frame	
Good quality		21-SEP-26

CUSTOMER_N PRODUCT_NAME		RATIN

CUSTOMER_FEEDBACK		REVIEWDAT

ARUN	Classic Wooden Frame	
Excellent product		20-SEP-26

5. Retrieve reviews with ratings above 4.

```
SQL> SELECT *
2 FROM REVIEW
3 WHERE RATING > 4;
```

REVIEWI	CUSTOMERID	PRODUCTID	RATING	

REVIEWTEST				REVIEWDAT

REV0001	201	101	5	
Excellent product				20-SEP-26
REV0003	203	103	5	
Very nice frame				22-SEP-26
REV0006	206	106	5	
Excellent quality				25-SEP-26

REVIEWI	CUSTOMERID	PRODUCTID	RATING	

REVIEWTEST				REVIEWDAT

REV0009	209	109	5	
Beautiful frame				28-SEP-26

Example Query:

5. Calculate Average Product Ratings

Use aggregate functions to analyze ratings.

1. Calculate average rating for each product.

```
SQL> SELECT P.PRODUCT_NAME,  
2         AVG(R.RATING) AS AVERAGE_RATING  
3 FROM PRODUCT P  
4 JOIN REVIEW R  
5 ON P.PRODUCT_ID = R.PRODUCTID  
6 GROUP BY P.PRODUCT_NAME;  
  
PRODUCT_NAME                                AVERAGE_RATING  
-----  
Decorative Wall Frame                        3  
Premium Acrylic Frame                       4  
Large Collage Frame                         2  
Modern Photo Frame                          5  
Table Photo Frame                           4  
Classic Wooden Frame                        4  
Elegant Acrylic Frame                       5  
Designer Wall Frame                         5  
Family Collage Frame                        5  
Classic Table Frame                         4  
10 rows selected.
```

2. Count the number of reviews per product.

```
SQL> SELECT P.PRODUCT_ID,  
2         P.PRODUCT_NAME,  
3         ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING  
4 FROM PRODUCT P  
5 JOIN REVIEW R  
6 ON P.PRODUCT_ID = R.PRODUCTID  
7 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME  
8 HAVING AVG(R.RATING) = (  
9     SELECT MAX(AVG_RATING)  
10    FROM (  
11        SELECT AVG(RATING) AS AVG_RATING  
12        FROM REVIEW  
13        GROUP BY PRODUCTID  
14    )  
15 );  
  
PRODUCT_ID  PRODUCT_NAME                                AVERAGE_RATING  
-----  
109 Designer Wall Frame                      5  
101 Classic Wooden Frame                     5  
103 Family Collage Frame                     5  
106 Modern Photo Frame                       5
```

3. Find highest-rated products.

```
SQL> SELECT P.PRODUCT_ID,  
2         P.PRODUCT_NAME,  
3         ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING  
4 FROM PRODUCT P  
5 JOIN REVIEW R  
6 ON P.PRODUCT_ID = R.PRODUCTID  
7 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME  
8 HAVING AVG(R.RATING) > 4  
9 ORDER BY AVERAGE_RATING DESC;  
  
PRODUCT_ID  PRODUCT_NAME                                AVERAGE_RATING  
-----  
109 Designer Wall Frame                      5  
106 Modern Photo Frame                       5  
103 Family Collage Frame                     5  
101 Classic Wooden Frame                     5
```

4. Identify products with average rating above 4.

```
SQL> SELECT P.PRODUCT_ID,  
2      P.PRODUCT_NAME,  
3      ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING  
4 FROM PRODUCT P  
5 JOIN REVIEW R  
6 ON P.PRODUCT_ID = R.PRODUCTID  
7 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME  
8 HAVING AVG(R.RATING) > 4  
9 ORDER BY AVERAGE_RATING DESC;
```

PRODUCT_ID	PRODUCT_NAME	AVERAGE_RATING
109	Designer Wall Frame	5
106	Modern Photo Frame	5
103	Family Collage Frame	5
101	Classic Wooden Frame	5

6. Identify Highly Rated Products

Generate product rating reports.

Report 1: Product Rating Analysis Display: ● Product name ● Number of reviews ● Average rating

```
SQL> SELECT P.PRODUCT_NAME,  
2      COUNT(R.REVIEWID) AS NUMBER_OF_REVIEWS,  
3      ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING  
4 FROM PRODUCT P  
5 LEFT JOIN REVIEW R  
6 ON P.PRODUCT_ID = R.PRODUCTID  
7 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME  
8 ORDER BY AVERAGE_RATING DESC NULLS LAST;
```

PRODUCT_NAME	NUMBER_OF_REVIEWS
AVERAGE_RATING	
Classic Wooden Frame	1
5	

Family Collage Frame	1
5	
Modern Photo Frame	1
5	

PRODUCT_NAME	NUMBER_OF_REVIEWS
AVERAGE_RATING	
Designer Wall Frame	1
5	
Table Photo Frame	1
4	
Premium Acrylic Frame	1
4	

PRODUCT_NAME	NUMBER_OF_REVIEWS
AVERAGE_RATING	
Classic Table Frame	1
4	
Elegant Acrylic Frame	1
4	
Decorative Wall Frame	1
3	

PRODUCT_NAME	NUMBER_OF_REVIEWS
AVERAGE_RATING	
Large Collage Frame	1
2	

10 rows selected.

Report 2:

Customer Feedback Analysis Find:

- Most reviewed products

```
SQL> SELECT P.PRODUCT_NAME,
2      COUNT(R.REVIEWID) AS REVIEW_COUNT
3 FROM PRODUCT P
4 JOIN REVIEW R
5 ON P.PRODUCT_ID = R.PRODUCTID
6 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
7 ORDER BY REVIEW_COUNT DESC;
```

PRODUCT_NAME	REVIEW_COUNT
Decorative Wall Frame	1
Designer Wall Frame	1
Classic Table Frame	1
Large Collage Frame	1
Modern Photo Frame	1
Premium Acrylic Frame	1
Elegant Acrylic Frame	1
Family Collage Frame	1
Table Photo Frame	1
Classic Wooden Frame	1

10 rows selected.

• Highly rated products

```
SQL> SELECT P.PRODUCT_NAME,
2      ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING
3 FROM PRODUCT P
4 JOIN REVIEW R
5 ON P.PRODUCT_ID = R.PRODUCTID
6 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
7 HAVING AVG(R.RATING) >= 4
8 ORDER BY AVERAGE_RATING DESC;
```

PRODUCT_NAME	AVERAGE_RATING
Classic Wooden Frame	5
Family Collage Frame	5
Designer Wall Frame	5
Modern Photo Frame	5
Table Photo Frame	4
Elegant Acrylic Frame	4
Classic Table Frame	4
Premium Acrylic Frame	4

8 rows selected.

```
SQL> SELECT P.PRODUCT_NAME
```

• Products requiring improvement

```
SQL> SELECT P.PRODUCT_NAME,
2      COUNT(R.REVIEWID) AS REVIEW_COUNT,
3      ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING
4 FROM PRODUCT P
5 JOIN REVIEW R
6 ON P.PRODUCT_ID = R.PRODUCTID
7 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
8 HAVING AVG(R.RATING) < 3
9 ORDER BY AVERAGE_RATING;
```

PRODUCT_NAME	REVIEW_COUNT	AVERAGE_RATING
Large Collage Frame	1	2

```
SQL> SELECT COUNT(*) AS FIVE_STAR_RATINGS
```

```
2 FROM REVIEW
3 WHERE RATING = 5;
```

```
FIVE_STAR_RATINGS
```

Report 3: Rating Distribution

• Number of 5-star ratings

```
SQL> SELECT COUNT(*) AS FOUR_STAR_RATINGS
2 FROM REVIEW
3 WHERE RATING = 4;
```

```
FOUR_STAR_RATINGS
-----
4
```

• Number of 4-star ratings

```
SQL> SELECT COUNT(*) AS FIVE_STAR_RATINGS
2 FROM REVIEW
3 WHERE RATING = 5;
```

```
FIVE_STAR_RATINGS
-----
4
```

• Number of low-rated products


```
SQL> SELECT COUNT(*) AS LOW_RATED_PRODUCTS
2 FROM (
3   SELECT PRODUCTID
4   FROM REVIEW
5   GROUP BY PRODUCTID
6   HAVING AVG(RATING) < 3
7 );
```

LOW_RATED_PRODUCTS
1

- Complete Rating

```
SQL> SELECT RATING,
2         COUNT(*) AS NUMBER_OF_RATINGS
3 FROM REVIEW
4 GROUP BY RATING
5 ORDER BY RATING DESC;
```

RATING	NUMBER_OF_RATINGS
5	4
4	4
3	1
2	1